Motion Engineering with Opus 5.5

Build Launch Videos You Can Sell

beamnwx

Independent creator / @beamnxw

Abstract

A launch video becomes easier to direct when its message, product action, and motion are expressed as editable events. This paper develops a production protocol for building such videos with Opus 5.5 and Remotion. It connects reference analysis, a five-part brief, frame-driven animation, staged review, synchronized audio, export verification, and a bounded commercial offer. An illustrative 20-second export demonstration specifies a 1080 by 1080 composition at 30 frames per second, including exact entrance, press, reveal, and reading intervals. The contribution is an operational method and executable specification; the example supplies design choices rather than measured performance or sales results.

1 From a product action to a deliverable

Tell Opus when the card should land, when the product should respond, and how long the headline should remain readable. Store those decisions with the assets and copy so a revision can change one event without rebuilding the whole film.

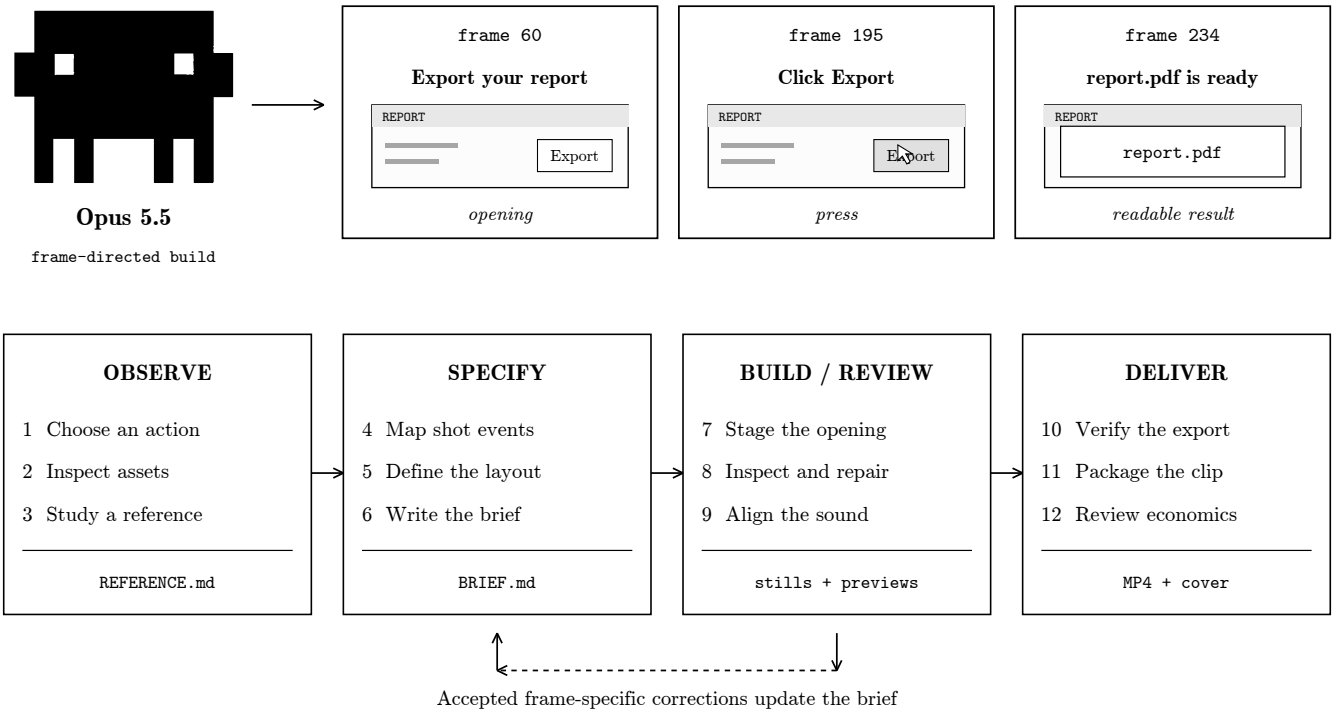


Figure 1. Opus directs a visible product action within the complete production workflow. Reference observations become a scene specification; accepted corrections return to that specification. Delivery includes the approved film and its agreed scope.

Reading path: sections 2-10 explain the workflow; Appendix A contains the analysis and builder prompts; Appendix B is the complete worked brief; Appendix C is the handoff prompt to use with this paper.

2 Analyze a reference for a specific decision



Figure 2. A reference contact sheet with bounding boxes and a landing path. These drawings are illustrative. Use actual source timestamps and consecutive source frames to measure the entrance and hold.

2.1 Give the reference a job

Choose a reference for a controlled landing, a transition that keeps the product visible, or a cursor action that makes a result obvious. Follow the main object from its first visible position to its destination.

Record what moves first, where the eye should land, when text becomes readable, and which state survives the cut. Use your own logo, words, product screen, and available typeface.

Sampling and event measurements

Two images per second provide an overview. Extract consecutive source frames around the entrance, landing, press, and cut before assigning exact event timing. Sparse samples can miss both overshoot and the true cut.

For each important interval, record the source frame rate, timestamps, object bounds, movement direction, duration, scale, opacity, and visible overshoot. Keep source timing separate from the output timeline.

An uncertain estimate should name the additional frames needed to verify it. Save observations in REFERENCE.md; Appendix A provides the analysis prompt.

2.2 Keep evidence identifiable

Label	What belongs here
Measured	Tool output, timestamps, verified pixel bounds.
Estimated	Visual judgment about a curve or a spatial anchor.
Proposed	Your new layout, copy, movement, and shot timing.

Claude can estimate spatial relationships, but its coordinates are approximate [3]. Check important positions against the extracted images, using image tools and pixel measurements where available.

Record the dimensions of the image actually inspected. An overview resized to 960 pixels wide does not have the coordinate system of the original video. Convert verified bounds back to source dimensions.

```
source_x = viewed_x * source_w / viewed_w
source_y = viewed_y * source_h / viewed_h
```

Then map the supplied product screen into your own card. The button and result anchors must be measured after that fit. A reference movement supplies a design decision, not a substitute for your assets.

3 Specify the frame before adding movement

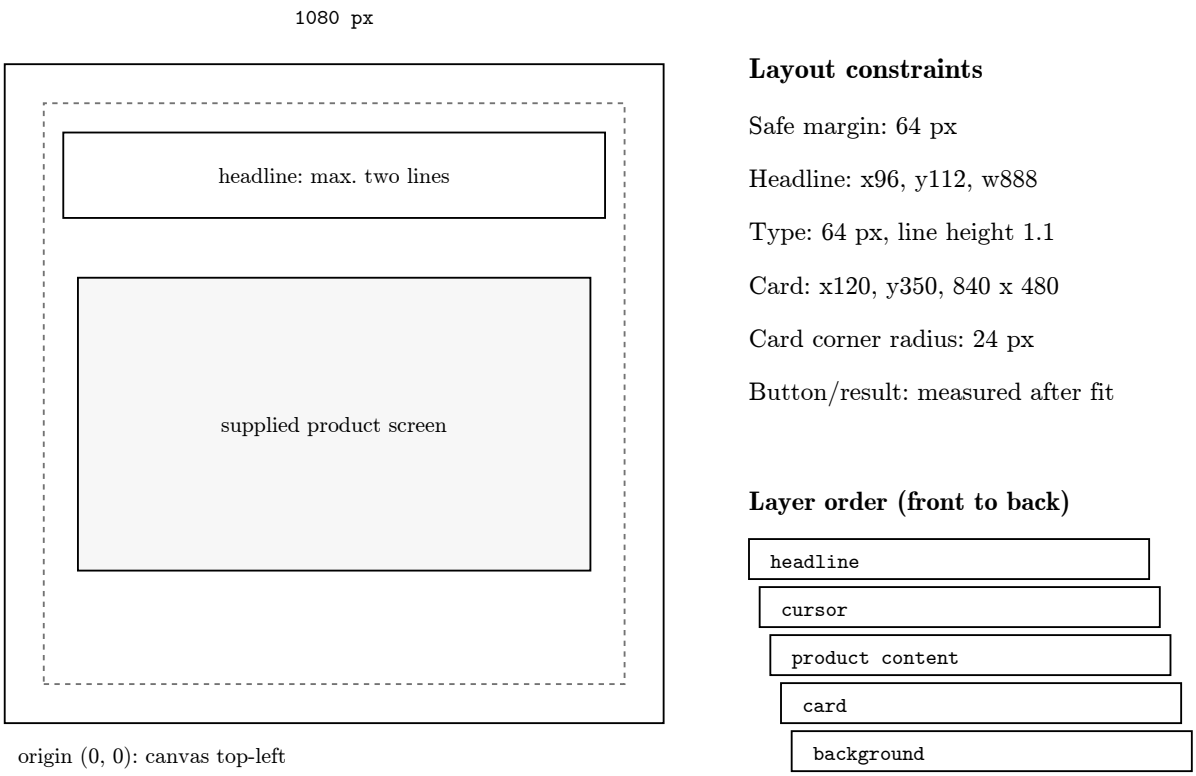


Figure 3. Canvas blueprint and layer stack for the worked example. The paper uses monochrome diagrams; the source brief retains its named background, text, and accent colors. Coordinates are proposed design values, not measurements of a reference.

3.1 Five decisions in BRIEF.md

The goal names one product action, its viewer, the opening outcome, and the closing destination. Delivery defines the composition, dimensions, frame rate, duration, and required assets.

Layout fixes the coordinate origin, safe area, type limits, fitted screen, measured interaction anchors, and layer order. The timeline specifies initial and final states, event frames, curves, and reading holds.

Implementation and review define deterministic rendering, editable constants, fixed seeds, the first stage, required stills, and feedback checkpoints. Keep all five parts together; Appendix B contains the full example.

3.2 Test the longest copy

Render the longest headline you expect to reuse with the actual font, weight, spacing, and deliberate line breaks. If it wraps to three lines, repair copy or layout before tuning motion.

Shorten the text, widen the box, or give supporting explanation another shot. Preserve the line limit, logo reservation, and settled positions in the brief.

Every copy change requires a new landing-frame check. Apply the same rule to the closing action when a longer product name or destination replaces a short one.

4 Write the shot as an event sequence

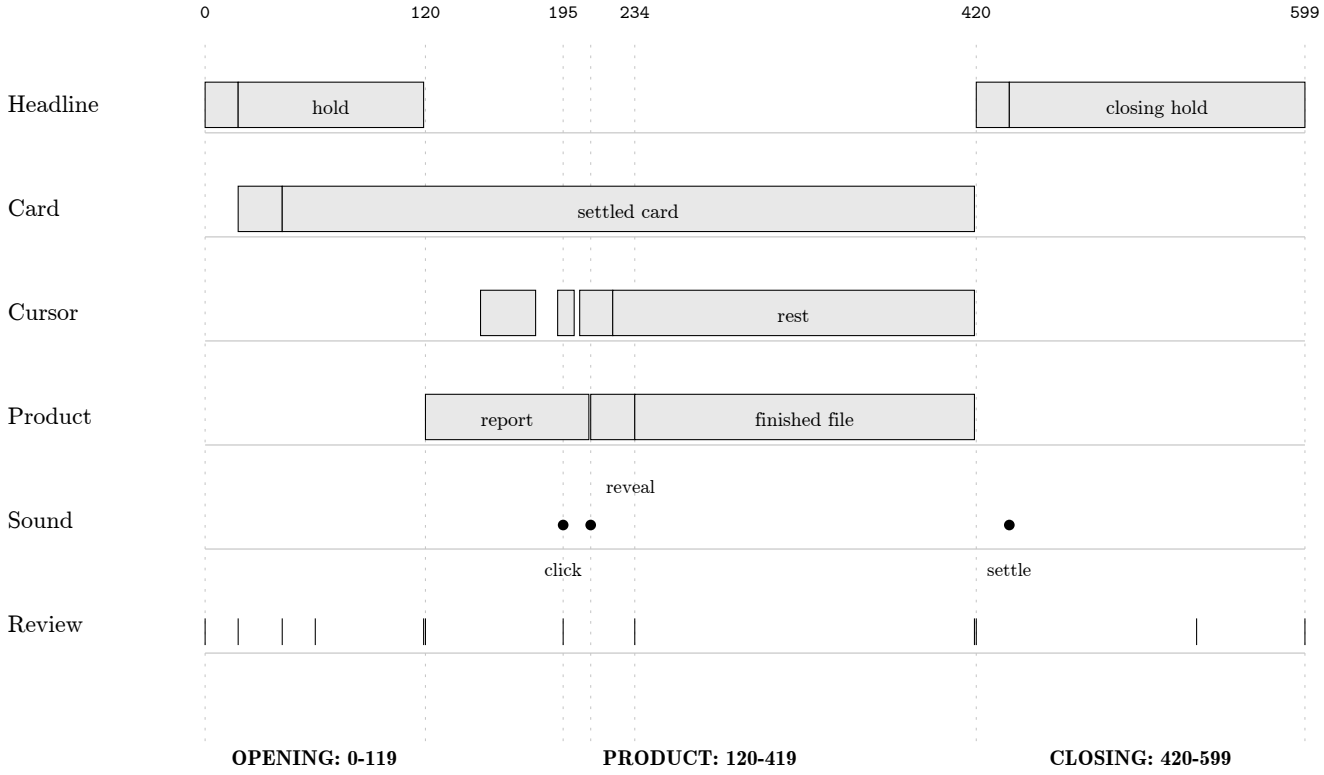


Figure 4. Motion score for the 600-frame composition. Narrow bars are changes; wide bars are settled holds. The product result becomes fully readable at 234. Review ticks refer to selected global frames, not every required intermediate check.

4.1 A visible cause and result

Show the starting state, move the cursor onto the action, register the press, and reveal the result. In the worked shot, a report becomes an exported file. Keep the finished name visible before the next cut.

Time	Frame	Event
4.0 s	120	Report settled
5.0 s	150	Cursor approaches
6.0 s	180	Cursor reaches Export
6.5 s	195	Press registers
7.0 s	210	File appears
7.8 s	234	Label fully readable
14.0 s	420	Closing begins

4.2 Frame conventions

For a chosen frame rate, convert seconds to frames using f = round(t x fps). State the rounding rule for a non-integer result. At 30 fps, 6.5 seconds corresponds to global frame 195.

The example uses inclusive global intervals: 0 through 599 contains 600 frames and lasts 20 seconds. A render range 90 through 149 contains 60 frames and lasts two seconds.

```
frame_count = end - start + 1
duration_s = frame_count / fps
global_frame = sequence_start
             + local_frame
```

Local frame 30 inside a Sequence starting at global 120 is global 150 [7]. A 0.5-second move is 15 frames in this 30 fps output and 30 frames in a 60 fps source. Keep both frame rates labeled.

5 Make every frame reproducible

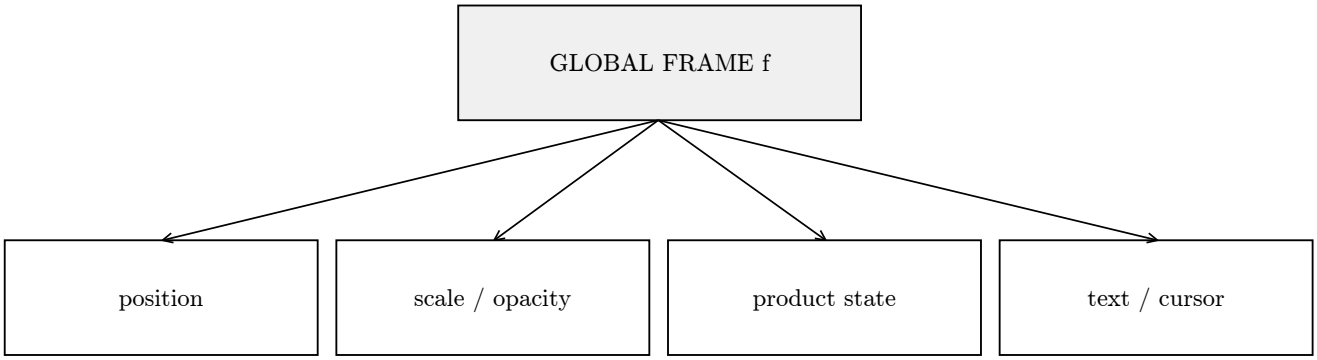


Figure 5. A frame defines the scene state. Store timing and geometry in shared constants; choose fixed seeds for variation so previews and exports reproduce the same moment [4-6, 16].

5.1 Derive motion from the frame

Remotion renders React components at the requested frame [4]. Use useCurrentFrame() with interpolation or spring() to compute position, scale, and opacity. Independent CSS transitions can desynchronize rendered frames [5].

Write timing in seconds first, convert it to frames once, and store offset, destination, duration, and landing settings together. A 0.6-second entrance takes 18 frames at 30 fps.

```
const f = useCurrentFrame();
const y = interpolate(
  f, [18, 42], [60, 0],
  {
    extrapolateLeft: "clamp",
    extrapolateRight: "clamp",
    easing: Easing.out(Easing.cubic),
  }
);
```

This excerpt implements the proposed card offset. Import the named Remotion APIs in your component; use the same frame interval to compute opacity. Expose these values as constants for later revision.

5.2 Direct the landing

The worked opening uses cubic ease-out with clamped endpoints. For a different landing, tune a spring with duration, damping, stiffness, and overshoot controls [6]. Keep its destination fixed while changing one parameter.

Introduce the main object before supporting information. Try a 0.1-second delay for a label, then tune the order in normal-speed playback. Hold secondary motion quiet while the headline is being read.

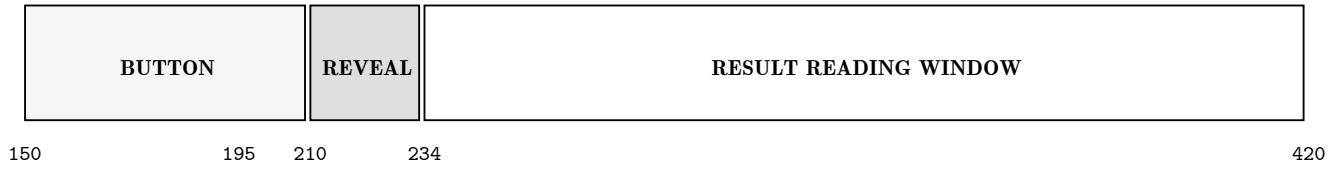
For a subtle push, specify scale 1.00 to 1.03 over three seconds. Interpolate between those endpoints. Repeated percentage multiplication compounds across frames and changes with frame rate.

Reusable state and boundaries

Give each scene incoming and outgoing position, scale, and highlighted fields. A continuous transition inherits the previous settled state. Share fonts, palette, spacing, and recurring elements. Work on one scene or a small related group, giving each task its assets, purpose, and boundary states.

Overlapping transitions reduce the sum of scene durations by their overlap [8]. After a local edit, inspect both neighboring boundaries. Reuse accepted entrance, cursor, and text components, then retune them for new copy and assets.

6 Allocate attention and a readable hold



Readability begins at 234, after the reveal has settled

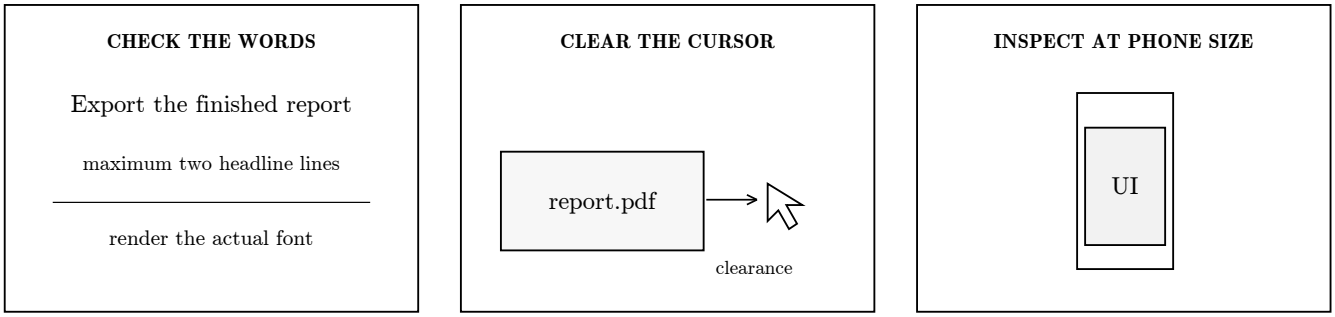


Figure 6. Attention follows the action before moving to the result. The readable hold is 234 through 419 inclusive: 186 frames, or 6.2 seconds. Typography, cursor clearance, and phone-size review are separate checks.

6.1 Count from legibility

Count the reading window from the first frame where the words can be read. Eight frames at 30 fps give about 0.27 seconds. That interval can be too short even when an entrance looks smooth.

Play the scene at normal speed and read aloud. Extend a hold that ends before the line is finished. Check interface labels at phone size; a large preview can hide an illegible product crop.

Keep attention on the button until its response appears, then give the result the reading window. If the cursor crosses a headline during an entrance, delay one event or change the path.

6.2 Use blur only where it helps

For a fast card entrance, decide what blur should contribute. Bound it to that entrance, let it settle with the movement, and keep the reading hold clear [11].

Inspect the sampled states blended by motion-blur tools for smeared type, persistent trails, or contamination across a cut. Render the entrance and its adjacent boundary, then replay them.

Motion has a job within each interval: entrance introduces an object, cursor movement identifies an action, hold supports reading, and transition carries the next state. Remove movement that competes with the current focal point.

7 Prepare the workspace and build in stages

7.1 Install the tools and select the model

Install Node.js, Claude Code, FFmpeg, and FFprobe before project creation. Use Claude Code 2.1.280 or later for Opus 5.5 and confirm model access [2]. The Remotion installer adds guidance for the coding agent you select [1].

Project creation and preview

```
npx create-video --yes --blank launch-video
cd launch-video
npx remotion skills add
npm run dev

# In a second terminal inside launch-video:
claude --model claude-opus-5-5
```

7.2 Put the inputs in place

```
launch-video/
  BRIEF.md
  REFERENCE.md
  reference.mp4
  public/
    logo + product screens
    font + supplied audio
  frames/
  out/
  src/
```

Inspect asset dimensions and font loading. Fit the real product screen before measuring the button or result. Ask for a missing screen or an illegible crop before implementation.

Create frames/ and out/ if they are absent. A reference is optional when you can specify the movement directly; never substitute a guessed measurement for an unavailable source.

7.3 Make the opening reviewable

Read BRIEF.md and the installed Remotion guidance. Summarize the layout, event frames, and curves; identify conflicting requirements or missing inputs. Implement the opening first.

Render global frames 0, 18, 42, 60, and 119 plus its moving preview. Check wrapping, margins, font, product legibility, and settled positions. Return the files and checks, then obtain feedback on this stage.

After direction is approved, build the product and closing scenes. Preserve card geometry at 120. Inspect frames 120, 195, 234, 419, 420, 540, and 599, plus intermediate states around interactions.

Reference overview extraction

```
ffmpeg -i reference.mp4 -vf "fps=2,scale=960:-2" frames/ref-%04d.jpg
```

The command samples two images per second at 960 pixels wide, preserves aspect ratio, and chooses an even output height [13]. Consecutive source frames around an event require a separate extraction at the source frame rate.

8 Review images, repair locally, then replay

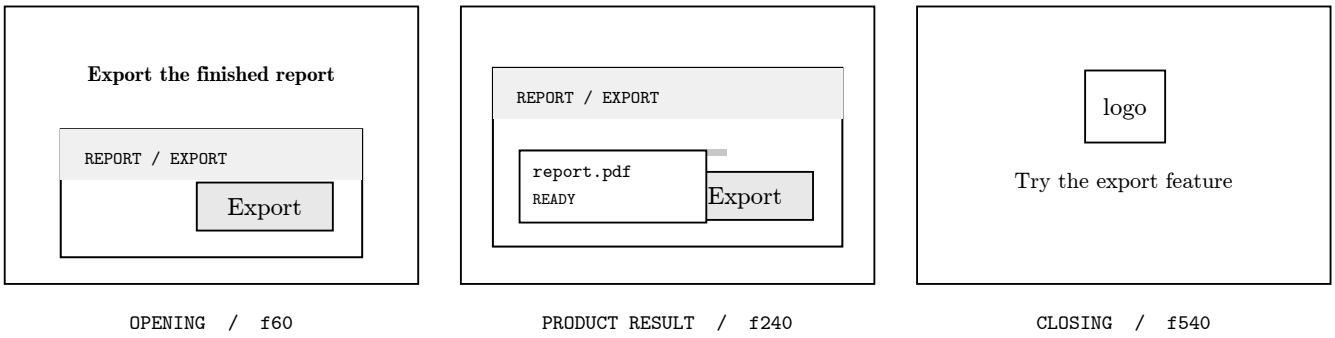


Figure 7. Three direction stills establish the message, visible result, and closing action before more motion detail is added. Stills complement normal-speed playback and boundary inspection.

```
npx remotion still Launch out/opening.png --frame=60
npx remotion still Launch out/product.png --frame=240
npx remotion still Launch out/closing.png --frame=540
```

8.1 Inspect the agreed moments

Move the product-result frame if your own reveal lands elsewhere [9]. Check main-text readability, visible product result, crop and overlap, and closing space.

Render 90 through 149 to inspect the opening-to-product boundary [10]. Review its two-second interval at normal speed. A local-frame label must also name the sequence offset.

Repair missing information and readability first, then timing, then visual detail. An independent critic receives the brief, images, and source frames; accept up to five specific corrections.

8.2 Four fields for a correction

Field	Example
Location	Global frame 210
Observed defect	Cursor overlaps result label
Change	Departure 204-210; 24 px clearance
Preserve	Press 195, reveal 210, card, copy, sound

Move the cursor to a resting point inside the card and keep its whole path clear of the label. Hold through 419. Render 195, 210, 222, and 240 plus a preview of 180 through 249; check intermediate frames for collisions or jumps.

```
npx remotion render Launch out/transition.mp4 --frames=90-149 --codec=h264
```

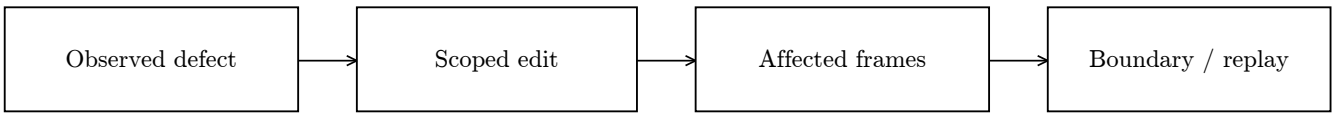


Figure 8. Local repair loop. Update only the implicated values, check their affected frames and neighboring boundary, and confirm the edit at normal speed. Appendix A contains the complete cursor-repair prompt.

9 Synchronize sound and verify the exported film

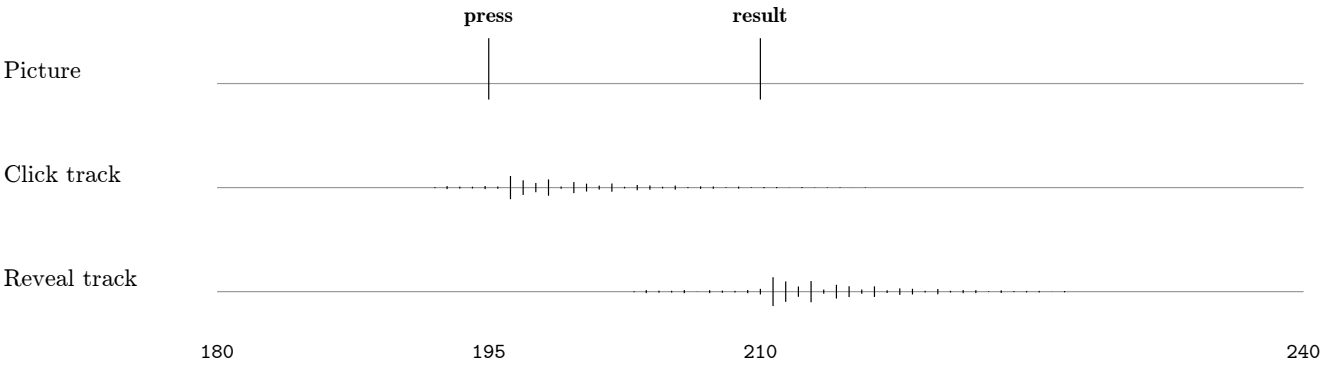


Figure 9. Illustrative cue alignment, not a measured waveform. Align the audible transient with the visual event. A file start can precede its strongest hit because of silence or a soft lead-in.

9.1 Place the audible event

Locate the click transient and reveal hit with waveform inspection and playback. Offset each clip so the audible event lands on the visible press at 195 or reveal at 210.

Event	Global frame
Button press / click	195
Result reveal / accent	210
Closing hold / music settles	438

Keep music and effects in separate tracks with editable placement and volume. For new audio code, Remotion recommends Audio from @remotion/media [12]. Check the loudest event, quietest useful cue, and ending through headphones and a phone speaker.

9.2 Inspect the file you deliver

After picture and sound are accepted, render the complete H.264 composition. Watch the exported file at normal speed, following the opening message, action, result, and final destination.

Check dimensions, average frame rate, and duration with FFprobe [14]. This brief expects 1080 by 1080 pixels, 30/1 fps, and approximately 20 seconds. Also inspect the end for unintended silence, truncated cues, or a frozen final event.

Archive the accepted export with BRIEF.md, scene timing, audio cues, and the relevant review files. A successful render command alone does not establish that the message or interaction is readable.

Final render and metadata inspection

```
npx remotion render Launch out/launch.mp4 --codec=h264

ffprobe -v error -select_streams v:0 -show_entries stream=width,height,avg_frame_rate,duration -of json out/launch.mp4
```

The companion commands.txt contains these executable commands. Keep the approved export and a cover still as the delivery files.

10 Package the first product: the Launch Clip

Package the worked exercise for a developer launching one real feature. The buyer receives a film for a launch post or product page. Build a sample with your own assets or with the product owner's permission before offering the package.

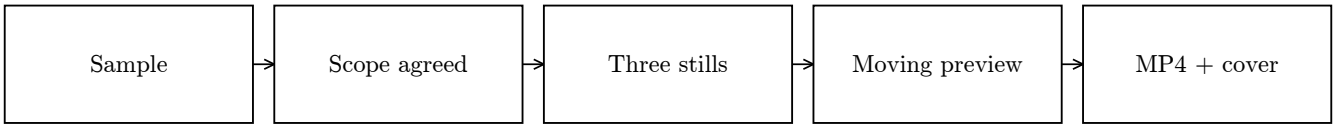


Figure 10. Commercial delivery sequence. Agree the script and scope before production, approve visual direction with stills, then accept the moving preview and final export.

10.1 Make the sample inspectable

Show the export beside its opening, action, and closing stills. Time asset preparation, feedback, rendering, and delivery. These hours determine whether the package is sustainable.

Choose a prospect with a working feature and a release date. Identify one action their audience needs to understand. A specific proposal can make the export feature the entire 20-second clip.

Share the sample and ask which feature they want to show. Confirm the script, assets, deadline, payment terms, revision scope, usage, and portfolio permission before starting.

10.2 Bound the included work

Include one feature, one approved script, a 20-second 1080 by 1080 MP4, one cover image from an approved frame, and one revision round covering timing, copy, and sound.

Quote additional formats, scenes, and project source files separately. Tie feedback to frame numbers so the revision round has a concrete edit list. Check the Remotion license for the version and organization involved [15].

A $150 pilot with $20 direct costs leaves $130 before tax and overhead. Four hours of work imply $32.50 per hour. These are illustrative inputs; replace them with actual cost and time.

Draft offer to adapt

```
Launch Clip | $150 illustrative pilot

Send your product screens, logo, and the feature you are launching.
I will turn one product action into a 20-second launch video.

Includes: a short script for approval, one square MP4, one cover image,
and one revision round within the approved script and visual direction.

We agree on the delivery date after reviewing your assets.
Extra formats, additional scenes, and project source files are quoted separately.
```

If revisions push the job beyond the planned hours, adjust the next quote or tighten scope. The sample calculation is neither a market rate nor an observed revenue result.

Appendix A Analysis, build, and repair prompts

Provide the real reference file for analysis and real product assets for implementation. The prompts below preserve the source article's staged review. Verify spatial estimates using the inspected frame dimensions and source-coordinate conversion in section 2.

A.1 Reference analysis

```
Analyze the main product reveal in reference.mp4.
First report its dimensions, frame rate, and duration using available tools.
Extract overview images, then consecutive frames around the entrance and landing.
Inspect the extracted images with image tools.

Save REFERENCE.md with:
Source timestamps and frame numbers for entrance, landing, hold, and cut.
Object bounding boxes at start and landing, in source-image coordinates.
Movement direction, duration, scale change, opacity change, and visible overshoot.
The order in which product, headline, and supporting labels appear.
The readable hold and the state carried into the next shot.

Label measured values, visual estimates, and proposed creative choices separately.
For an uncertain value, identify the frames needed to check it.

Use our assets to propose one original export-demonstration shot.
Specify its layout, focal point, entrance, product response, and reading hold.
List missing assets and questions requiring approval.
Return this shot plan before implementation.
```

A.2 First implementation stage

```
Read BRIEF.md and the installed Remotion guidance.
Inspect public/ and confirm that the required assets are available.
Summarize the opening's layout, event frames, and animation curves.
Flag any conflicting requirements or missing inputs before implementation.
Implement only the first review stage specified in BRIEF.md.
Return the stills, moving preview, changed files, and checks performed.
Wait for approval before continuing.
```

A.3 Scoped cursor-path repair

```
Revise the Launch composition using BRIEF.md.
Observed defect: the cursor overlaps the result label at global frame210.

Change only the cursor's post-press path:
Start the departure at204 and reach its resting point by210.
Choose the resting point inside the card, with 24 pixels of clearance from the result label.
Use a path that stays clear of the label throughout the move.
Preserve the press at195, reveal at210, card geometry, copy, and audio cues.
Hold the cursor at its destination through419.

Render stills at195,210,222,240 and a preview of180 through249.
Check intermediate frames for overlap and jumps.
Report the changed path coordinates and output filenames for review.
```

Appendix B Complete worked scene brief

Replace the bracketed fields with your assets and approved copy. These coordinates, colors, and timing are starting choices for this exercise. The color values belong to the source production brief; all paper diagrams are monochrome.

Save as BRIEF.md in the Remotion project

```
GOAL
Demonstrate how [product] exports a finished report.
Viewer: [who uses this feature].
Opening copy: [outcome, maximum eight words].
Closing copy: [one next action and destination].

DELIVERY AND ASSETS
Composition: Launch, 1080 by 1080, 30 fps, 600 frames.
All timeline numbers below are global, inclusive frame numbers.
Assets under public/: [logo], [report screen], [exported-file screen], [font], [audio].
Inspect asset dimensions and font loading before implementation.
Request missing screens or illegible crops before proceeding.

LAYOUT
Coordinates use the canvas top-left as origin; units are pixels.
Background #101318, primary text #F5F7FA, accent #8AD8B0.
Keep essential content inside a 64-pixel outer margin.
Headline box: x96, y112, width888; 64-pixel font, 1.1 line height, maximum two lines.
Product card: x120, y350, width840, height480, corner radius24.
Fit the supplied screen while keeping the export control readable.
Record button and result anchors from the fitted screen.
Layer order: background, card, product content, cursor, headline.

OPENING: FRAMES 0 THROUGH 119
Headline enters during 0 through 18: vertical offset24 to0, opacity0 to1.
Card enters during 18 through 42: vertical offset60 to0, opacity0 to1.
Use cubic ease-out with clamped endpoints for both entrances.
Hold the settled layout during 42 through 119.

PRODUCT: FRAMES 120 THROUGH 419
Carry the card into frame120 at its exact settled position and scale.
Move the cursor to the measured button anchor during 150 through 180.
Choose a path clear of text and record its waypoints.
Press at195; animate the cursor scale1 at192, to0.92 at195, back to1 at201.
Show the button response at195 using supplied assets or an approved accent overlay.
Move the cursor toward a clear resting point during 204 through 222.
Show the supplied exported-file state at210; finish its reveal by234.
Keep the result readable and stationary through419.

CLOSING: FRAMES 420 THROUGH 599
Cut to the supplied logo and closing copy at420.
Fade the closing copy in during 420 through 438; hold through599.

IMPLEMENTATION AND REVIEW
Drive animated values from useCurrentFrame; expose timing and layout constants.
Use fixed seeds for any variation; keep the camera stationary for this pass.
If audio is supplied, align its audible click with195 and reveal hit with210.
First build the opening and render frames0,18,42,60,119 plus its moving preview.
Check text wrapping, margins, screen readability, and settled positions.
Report changed files, outputs, and unresolved issues; wait for feedback.
After approval, build the remaining scenes and inspect frames120,195,234,419,420,540,599.
```

Frame ranges are global and inclusive. Fit the supplied screen before choosing button and result anchors. Appendix A.3 deliberately changes the post-press departure from 204-222 to 204-210 when a collision is observed.

Appendix C Handoff prompt for Opus 5.5

Attach this PDF and the product assets, then replace the input fields. A tool-enabled coding session can implement and render the scenes; a text-only chat can prepare the brief and shot plan. The opening and moving-preview checkpoints remain explicit.

Copy into a tool-enabled Opus 5.5 session

```
Use the attached Motion Engineering with Opus 5.5 paper as the production protocol.
Work in Claude Code with local file, image, terminal, and Remotion tools.
If this chat cannot write files or render video, explain the missing tool access and
produce the brief and shot plan only. Do not claim to have rendered an output.

PROJECT INPUTS
Product: [name and one-sentence description]
Feature to demonstrate: [one real action and its visible result]
Viewer: [who needs this feature]
Opening copy: [outcome, at most eight words]
Closing copy and destination: [one next action]
Assets: [logo, starting product screen, finished-result screen, font, optional audio]
Reference: [optional reference.mp4 and the movement decision to study]
Output: Launch, 1080 x 1080, 30 fps, 600 frames, H.264 MP4 and one cover still.

1. INSPECT AND SPECIFY
Read the paper and the installed Remotion guidance. Inspect the workspace and
actual assets. Confirm dimensions, font loading, crop quality, and model/tool access.
Ask for missing essential inputs; label estimates and proposed creative choices.
If a reference is supplied, probe its metadata, extract overview and consecutive
source frames around events, inspect them, and save REFERENCE.md. Record viewed
image dimensions, source dimensions, source fps, timestamps, verified bounds,
movement, appearance order, overshoot, readable holds, and outgoing states.
Save BRIEF.md with the five parts in section 3. Use Appendix B as a starting
specification and adapt it to the real product screens. Measure button and result
anchors after fitting the screen. Keep output global frames separate from source
and scene-local frames. Expose layout, event timing, curves, and audio cues.

2. BUILD THE FIRST REVIEW STAGE
Create or reuse a Remotion project with composition id Launch. Install the agent
skills when needed and within the authorized workspace. Derive animation from
useCurrentFrame(), interpolation or spring; use fixed seeds and clamped endpoints.
Implement only the opening first. Test the longest headline and actual font.
Render frames 0, 18, 42, 60, 119 and a moving opening preview. Inspect readability,
wrapping, margins, clipping, fitted screen, and final positions. Fix defects locally.
Return BRIEF.md, REFERENCE.md if applicable, changed files, output paths, and checks.
Stop at this review checkpoint and wait for approval of the direction.

3. COMPLETE AFTER OPENING APPROVAL
Build the product action and closing. Preserve card continuity at 120; cursor
approach 150–180; press 195; visible result 210; fully readable result 234;
closing starts 420, settles 438, ends 599. Adapt these values only with an updated
brief and a stated reason. Preserve a clear cause-and-result sequence.
Render the three direction stills at 60, 240, 540; inspect event frames and both
scene boundaries. Apply accepted corrections as location, defect, change, and
invariants. Recheck intermediate frames and normal-speed playback after each repair.
Align the audible click with 195 and reveal transient with 210 when audio is supplied.
Inspect the waveform, separate effects/music tracks, and check the ending.
Return the moving preview for picture and sound approval before final export.

4. EXPORT AND HAND OFF
After preview approval, render the full H.264 file. Watch the exported MP4 and probe
its dimensions, average fps, and duration: 1080 x 1080, 30/1, approximately 20 s.
Deliver the accepted MP4, one cover, approved brief, cue list, commands, and a concise
verification report. If a check cannot be performed, state that limitation.
For a commercial package, draft the bounded Launch Clip scope in section 10 and
replace illustrative prices and costs with agreed inputs. Do not publish the video,
send a pitch, accept terms, spend money, or claim a sale without my authorization.
```

References

[1] Remotion. Coding agents: project setup and agent skills.
https://www.remotion.dev/docs/ai/coding-agents

[2] Anthropic. Claude Code model configuration.
https://code.claude.com/docs/en/model-config

[3] Anthropic. Vision: image processing and spatial limitations.
https://platform.claude.com/docs/en/build-with-claude/vision

[4] Remotion. The fundamentals.
https://www.remotion.dev/docs/the-fundamentals

[5] Remotion. Animating properties.
https://www.remotion.dev/docs/animating-properties

[6] Remotion. Spring settings.
https://www.remotion.dev/docs/spring

[7] Remotion. Sequence and frame offsets.
https://www.remotion.dev/docs/sequence

[8] Remotion. TransitionSeries and duration accounting.
https://www.remotion.dev/docs/transitions/transitionseries

[9] Remotion. CLI still rendering.
https://www.remotion.dev/docs/cli/still

[10] Remotion. CLI rendering and frame ranges.
https://www.remotion.dev/docs/cli/render

[11] Remotion. Motion blur.
https://www.remotion.dev/docs/motion-blur

[12] Remotion. Audio from @remotion/media.
https://www.remotion.dev/docs/media/audio

[13] FFmpeg. Filter documentation: fps and scale.
https://ffmpeg.org/ffmpeg-filters.html

[14] FFmpeg. FFprobe metadata inspection.
https://ffmpeg.org/ffprobe.html

[15] Remotion. License terms.
https://github.com/remotion-dev/remotion/blob/main/LICENSE.md

[16] Remotion. Seeded random().
https://www.remotion.dev/docs/random

Source article and authorship

beamnwx. Motion Engineering with Opus 5.5: Build Launch Videos You Can Sell. Original article on the @beamnxw X account. The author's local article is the source for the worked brief and prompts.

https://x.com/beamnxw/status/2108616740846420354

This production protocol specifies an illustrative exercise. It reports no completed video benchmark, client sale, or measured conversion result. Prices and workload examples describe arithmetic assumptions to replace with actual inputs.